

온라인 강의 플랫폼

NestJS · React · Toss Payments · 운영형 풀스택

P1 → P2 → P3 단계별 구축 최종 보고

2026-06-29 · 최종 발표

발표 개요

1. 프로젝트 소개

- 무엇을 만들었나
- 사용 스택

2. 단계별 진행

- P1: 도메인 골격
- P2: 결제·진도율·관리자
- P3: 외부연동·관측성·CI/CD

3. P3 핵심 디테일

- Toss 결제 + 먹등성 + 재시도
- 스케줄러 + 작업 로그
- 헬스 / 메트릭 / 구조화 로그
- 보안 / 성능 / CI·CD

4. 데모 · 회고 · Q&A

한 줄 요약

NestJS + React + MySQL 로 만든 강의 플랫폼.

결제는 Toss, 알림은 이메일, 운영은 **/admin** 콘솔과 메트릭으로.

기능

- 강의 CRUD / 결제 / 수강 / 진도율 / Q&A / 관리자 콘솔

운영

- **/healthz** **/readyz**
/metrics
- 구조화 JSON 로그
- 운영 로그 (audit)
- 스케줄러 + 작업 이력

자동화

- GitHub Actions CI
- SSH + **git pull** CD
- 헬스체크 자동 검증

기술 스택

백엔드

- **NestJS 11** + TypeORM + class-validator
- **MySQL 8** (운영) / better-sqlite3 (테스트)
- JWT 인증, Role guard
- **@nestjs/schedule** (cron)
- nodemailer (이메일 알림)

프론트엔드

- **React 18 + Vite 6**
- React Router v6
- Toss Payments v2 SDK
- React.lazy 코드 스플리팅

외부 연동

- **Toss Payments API**
- SMTP (이메일)

운영 / 인프라

- docker + docker compose
- GitHub Actions (CI/CD)
- SSH 기반 배포 (GHCR 미사용)

관측성

- 인-프로세스 메트릭 (Prometheus 텍스트)
- JSON 구조화 로그

P1 — 도메인 골격

강의 · 사용자 · 수강 등록 의 최소 단위

P1: 도메인 / 인증

엔티티

- **Course** (강의)
- **User** (student / instructor / admin)
- **Enrollment** (수강)

인증

- JWT (HS256, 7일)
- 회원가입 / 로그인
- Role-based guard

확보한 것

- `@Roles('admin')` 데코레이터로 권한 매트릭스 명시
- `ValidationPipe(whitelist)` 로 unknown 필드 차단
- Repository 패턴 + class-validator

P2 — 결제·진도율·관리자

"동작하는 코드" 에서 "운영 가능한 코드" 로

P2 핵심 추가

결제 흐름

- 장바구니 → 결제 → 수강 자동 등록
- 먹등성 사고 도입 (중복 결제 방지)

진도율

- 강의에 `youtube_url`
- 수강자만 시청 가능
- 학생/강사/관리자 권한별 조회

Q&A 게시판

- 수강생/강사 작성
- 답변은 해당 강사 + 관리자만
- "답변 대기/완료" 배지

관리자 콘솔 (완전 분리)

- `/admin/*` 별도 다크 레이아웃
- 대시보드/사용자/결제/로그
- 운영 로그 (Audit log)

영상 없으면 완료 금지

- BE 400 + FE 버튼 비활성

P3 — 외부 연동 · 관측성 · CI/CD

"외부 시스템과 안정적으로 통신하고, 장애에 견디며, 운영자가 관찰·조작할 수 있는 서비스"

P3 무엇을 추가했나

외부 연동

- **Toss Payments** v2 (prepare → 위젯 → confirm)
- **이메일** (nodemailer)
- **재시도** (지수 백오프 + jitter)
- **멥등성** (`idempotency_keys`)
- **결제 영수증** + 영수증 메일

배치 / 스케줄러

- `@nestjs/schedule` cron 2종
- `job_runs` 이력 테이블
- 관리자 수동 실행 API

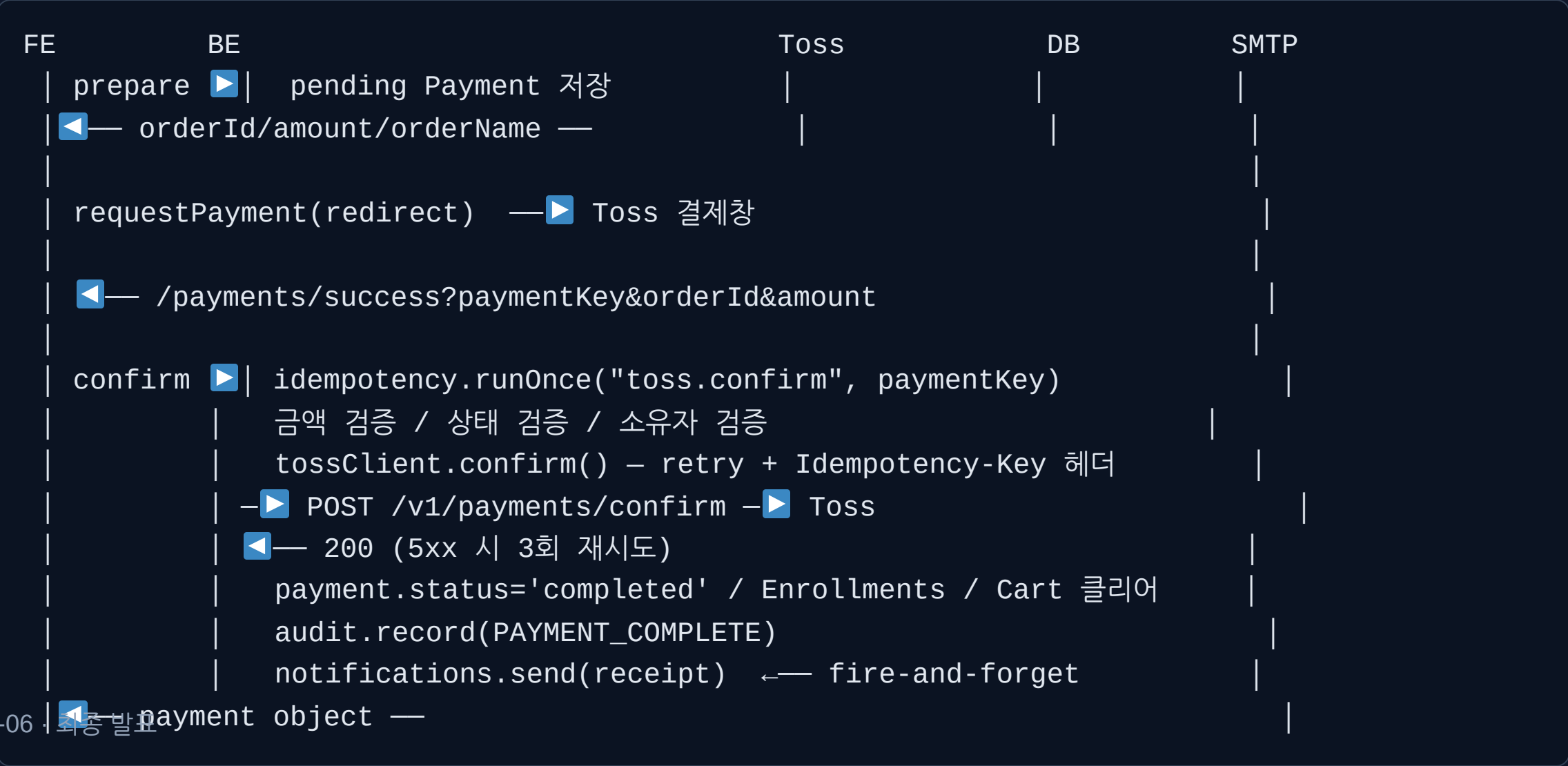
관측성

- `/healthz` (liveness)
- `/readyz` (DB ping)
- `/metrics` (Prometheus 텍스트)
- `x-request-id` 추적
- HTTP 인터셉터 JSON 한 줄 로그

보안 / 성능

- 시크릿 환경변수 + GitHub Secrets
- 인덱스 추가
(`uq_enrollment_user_course` 등)
- FE 코드 스플리팅 (278 → 191kB)

결제 시퀀스 (Toss)



멱등성 — 같은 paymentKey 두 번 와도 안전

정책

- 동일 paymentKey 호출 → 첫 결과 그대로 반환
- 두 번째 호출은 Toss API 가지 않음
- DB 의 idempotency_keys (key UNIQUE) 가 race 안전망

효과

- 사용자가 더블 클릭해도 안전
- 네트워크 재시도가 중복 결제 안 만듦
- Toss → 우리 → 사용자 모두 같은 결과

코드 한 곳에 응집

```
async tossConfirm(...) {  
  return this.idempotency.runOnce(  
    'toss.confirm',  
    paymentKey,  
    async () => this.confirmInternal(...)  
  );  
}
```

테스트로 검증

- ✓ Toss 502 두 번 → 3번째에서 성공
- ✓ 같은 paymentKey 재호출 → 멱등 응답 (DB에 추가 결제 X)

재시도 — 지수 백오프 + jitter

정책

- **5xx / 429 / timeout** → 재시도
- **4xx** → 즉시 실패
- **attempt N: $250\text{ms} \times 2^{(N-1)} + \text{jitter}(\leq 100\text{ms})$**
- 최대 3회

분류

```
function isTransient(err) {
  if (err.status >= 500) return true;
  if (err.status === 429) return true;
  if (err.code === 'ETIMEDOUT') return true;
  if (err.code === 'ECONNRESET') return true;
  // 4xx → false
}
```

2026-06 · 최종 발표

적용 지점

1. **TossClient.confirm()** — 결제 승인
2. **NotificationsService.send()** — 메일 송신

Idempotency-Key 헤더

재시도 중 같은 호출이 두 번 가도 Toss가 같은 결과 반환:

```
fetch(toss, {
  headers: {
    'Idempotency-Key': req.paymentKey,
    ...
  }
});
```

결제 영수증

발급

- `GET /payments/:id/receipt`
- 권한: 본인 + admin
- 미완료 결제 → 400

영수증 데이터

```
{
  "receipt_no": "R-000000042",
  "issued_at": "2026-06-17T...",
  "order_id": "ord_...",
  "payment_key": "...",
  "method": "카드",
  "buyer": { "name": "...", "email": "..." },
  "total_amount": 12000
}
```

2026-06-17 최종 발표

자동 이메일 발송

- 결제 완료 직후 백그라운드 송신
- `nodemailer` + 재시도 3회
- 메일 실패해도 결제는 성공 (비차단)

SMTP 미설정 시

- **stream transport (드라이런)** 으로 폴백
- 개발 환경에서도 동작
- 보낸 메일 로그로 검증 가능

스케줄러 + 작업 로그

등록된 cron 작업

작업	주기
<code>expire-stale-pending-payments</code>	매 시간
<code>cleanup-old-records</code>	매일 03:00

모든 실행 기록

`JobsService.runTracked(name, fn)` 으로
감싸 `job_runs` 에 자동 저장:

- 시작/종료 시각
- `duration_ms` / `processed` / `error`

관리자 API

- `GET /admin/jobs/runs?name=&status=`
— 이력
- `POST /admin/jobs/:name/run` — 수동
실행

운영 효과

- 30분+ `pending` 결제가 누적되지 않음
- 90일+ `job_runs` , 30일+
`idempotency_keys` 자동 청소
- 작업 실패는 다음 cron 에서 자연 재시도

관리자 콘솔 — 완전 분리

/admin/* 다크 레이아웃

- 공용 Navbar 숨김 (**MaybeNavbar** 가드)
- 좌측 사이드바 + 메인 영역
- "일반 사이트로 돌아가기" 명시적 링크
- 비관리자 접근 시 거부 카드

페이지

1. **대시보드** — 사용자/강의/결제/매출 카드
2. **사용자 관리** — 검색 + 권한 변경
3. **결제 내역** — 상태별 필터
4. **운영 로그** — action/날짜 필터

자동 기록되는 운영 액션

Action	의미
<code>user.register</code>	회원가입
<code>course.create/update/delete</code>	강의 CRUD
<code>user.role_change</code>	권한 변경
<code>enrollment.cancel</code>	수강 취소
<code>payment.complete</code>	결제 완료
<code>qna.question.create</code>	질문 작성
<code>qna.answer.create</code>	답변 작성

본인 admin 자기-강등 방지 등 안전장치 포함

관측성 — Health · Metrics · Logs

Liveness / Readiness

```
GET /healthz → { status: "ok" }
GET /readyz → {
  status: "ok",
  uptime: 3421,
  checks: { database: { ok: true } },
  version: "dev"
}
```

Prometheus 텍스트 메트릭

```
http_requests_total 123
http_request_errors_total 4
http_request_duration_ms_sum 8421
http_request_duration_ms_bucket{le="50"} 87
http_request_duration_ms_bucket{le="100"} 110
```

구조화 JSON 로그 (한 줄)

```
{
  "ts": "2026-06-17T...",
  "level": "info",
  "event": "http.request",
  "requestId": "07daf243-a729-...",
  "method": "POST",
  "url": "/api/payments/toss/confirm",
  "status": 201,
  "duration_ms": 173
}
```

x-request-id

- 미들웨어가 자동 생성 / 헤더에서 받아옴

보안

입력 / 권한

- `ValidationPipe(whitelist)` — unknown 필드 drop
- DTO + class-validator
- TypeORM 파라미터 바인딩 (SQL Injection 방어)
- Role guard + 본인 검증 (소유자 vs 관리자)

XSS 방어

- React 기본 escape
- 이메일 본문도 `escapeHtml()` 적용

시크릿 관리

위치	무엇이 들어감
Repo	코드 + <code>.env.example</code> 만
Docker 이미지	시크릿 절대 안 들어감
서버 <code>.env</code>	런타임 시크릿 (JWT/TOSS/SMTP/DB)
GitHub Secrets	SSH 접속 정보만

회전 정책

- JWT_SECRET 변경 → 전 사용자 재로그인
- 노출 의심 즉시 회전 + 감사 + 보고서

성능

DB 인덱스 추가

엔티티	인덱스	효과
enrollments	UNIQUE(user_id, course_id)	중복 차단 + 빠른 조회
enrollments	(course_id, completed_at)	완료자 수 집계
payments	(user_id, created_at), status, order_id	결제 화면, 스케줄러
audit_logs	actor_id, action, created_at	로그 필터
idempotency_keys	UNIQUE(key)	중복 confirm 차단
job_runs	name, status, created_at	작업 이력

FE 코드 스플리팅

Before

dist/assets/index-*.js 278 kB | gzipped

After (React.lazy + Suspense)

dist/assets/index-*.js 191 kB | gzipped
+ 18개 페이지별 chunk (각 2~10 kB)

→ 초기 진입 시 불필요한 코드
운로드 (Admin 콘솔/Q&A 등
시점에만)

측정

CI / CD

CI — `.github/workflows/ci.yml`

`main` / `P3` push, PR → 자동 실행

- **Backend job**: install / lint / typecheck / build / **E2E 83개**
- **Frontend job**: install / lint / build

CD — `.github/workflows/cd.yml`

`main` push → 자동 배포

1. sshpass 설치
2. SSH 로 EC2 접속
3. `git fetch && git reset --hard origin/main`

2026-06 · 최종 발표

GitHub Secrets — 2개만

Secret	설명
<code>SSH_HOST</code>	EC2 퍼블릭 IP
<code>SSH_PASSWORD</code>	SSH 비밀번호

선택 (기본값 있음): `SSH_USER` `SSH_PORT`
`DEPLOY_PATH` `DEPLOY_BRANCH`

런타임 시크릿은?

서버 `.env`에만 — GitHub 에 저장 X

- JWT_SECRET / TOSS_SECRET_KEY / SMTP_* / DB_PASSWORD

운영 매뉴얼 (Runbook 발취)

첫 5분 체크리스트

1. `curl /healthz` — 200?
2. `curl /readyz` — DB ok?
3. `curl /metrics.json` — error 추세
4. `docker compose ps` — 컨테이너 상태
5. `docker compose logs --tail=200 backend`

장애 시나리오별 대응

- DB 끊김 → `restart db`
- 응답 느림 → 슬로우 쿼리 확인
- `TOSS 5xx` → 자동 재시도, 상태페이지 확인

사고 보고서 양식

사고 한가운데서 새로 쓰지 않도록 양식 미리 작성:

- 요약 / 타임라인 / 원인 / 영향 / 액션 아이템 / 배운점

시크릿 노출 의심 시

1. 즉시 회전 — JWT / TOSS / SMTP / DB
2. 감사 — `git log -p .env*`
3. 사용자 공지
4. 사고 보고서 기록

테스트 — 83개 E2E

Suite	개수	영역
app.e2e-spec.ts	1	헬스
payment-flow.e2e-spec.ts	10	장바구니 → 결제 → 수강
progress-flow.e2e-spec.ts	16	진도율 + 권한
toss-payment.e2e-spec.ts	12	Toss 위젯 흐름 + 번조 차단
admin-qna-feedback.e2e-spec.ts	24	관리자/Q&A/영상없음 차단
p3-ops.e2e-spec.ts	20	retry / 먹등성 / 영수증 / 스케줄러 / health / metrics / unique 인덱스

커버하는 핵심: 먹등성 / 재시도 / 권한 매트릭스 / 번조 차단 / 영상 없으면 완료 금지 / 스케줄러 / 헬스 / `x-request-id` 헤더 echo / DB unique 인덱스

데모 시나리오

일반 사용자 흐름

1. 회원가입 (학생)
2. 강의 목록 → 상세 → 장바구니
3. 결제 → Toss 위젯 → 카드 결제
4. 자동 수강 등록 + 영수증 메일
5. "▶ 강의 수강" → YouTube + 진도율
6. Q&A — 질문 작성 → 강사 답변
7. 영상 없는 강의 → 완료 처리 시도 → 차단 확인

관리자 흐름

1. 관리자 로그인
2. `/admin` 다크 콘솔 진입
3. 대시보드 — 사용자/매출 카드
4. 사용자 관리 — 권한 변경 → 즉시 audit log 기록
5. 결제 내역 — 모든 결제 / 상태 필터
6. 운영 로그 — 자동 기록 액션 확인
7. 운영 작업 수동 실행 + 이력

운영 가시성

- `/healthz` `/readyz` `/metrics`

기술 의사결정 (ADR)

ADR-0001

Toss Payments + v2 SDK

- 문서/테스트 환경 우수
- v1 의 iframe redirect 보안 이슈 회피
- v1 키 호환

ADR-0002

스케줄러 = 인-프로세스 cron

- BullMQ + Redis 보류
- 트래픽/작업 규모 작음
- 마이그레이션 트리거 명시 (분당 100건+)

ADR-0003

배포 = SSH + git pull

- GHCR 미사용
- 단일 호스트, 빌드 시간 짧음
- 롤백이 `git checkout <sha>` 한 줄

공통 원칙

- 트레이드오프를 글로 남긴다
- 마이그레이션 조건을 미리 정한다
- "왜 안 했는가" 도 결정

학습 점수표 (자체 평가)

영역	P1 시작	P3 완료
도메인 모델링 (TypeORM, 관계)	2	4
트랜잭션 / 멍등성 사고	1	4
관측성 (메트릭/로그/헬스)	1	3
외부 API 안전 통신	2	4
운영 문서 작성	1	3
CI / CD 자동화	2	3

5점 만점. **트랜잭션·외부 통신·운영 사고**가 가장 크게 성장한 영역.

회고 — Keep / Problem / Try

잘한 결정 (Keep)

- 결제 confirm 에 **먹등성** 도입
- 영수증 메일을 **비차단**으로
- 관리자 콘솔을 **완전 분리**
- 외부 API 어댑터를 별도 클래스로 (테스트 격리 용이)

아쉬운 결정 (Problem)

- 진도율이 슬라이더 수동 저장 — YouTube IFrame API 로 자동화 여지
- 단일 인스턴스 가정에 묶여 큐/락 미도입

다음 단계 (Try)

1. **자동 진도율** — YouTube IFrame Player API
2. **환불/취소** — Toss **cancel** API
3. **알림 다양화** — 카카오톡, Slack
4. **BullMQ** 도입 — 알림/리마인더 늘어나면
5. **OpenTelemetry** — 메트릭/트레이스 표준화
6. **Playwright** — 결제 위젯까지 E2E

한 페이지 요약

전체 코드

- BE: 40+ 모듈 / 엔티티 11 / 컨트롤러 13
- FE: 페이지 19 (라우트 단위 lazy) / 코드 스플릿 18 chunk
- 문서: `docs/p3/` 7 + `docs/adr/` 3 + 회고/포트폴리오

검증

- E2E **83개 통과**
- 라이브 BE smoke test (`/healthz`, `/readyz`, `/metrics`, `x-request-id`) 정상
- FE 코드 스플리팅으로 초기 번들 278 → 191kB (gz 78 → 62kB)

자동화

- CI: lint / typecheck / build / e2e

- CD: GitHub Secrets **IP + 비밀번호** 등 개마 드로하며 SSH 자동 배포

감사합니다

질문 환영합니다 🙏

- Portfolio: `PORTFOLIO.md`
- Architecture: `docs/p3/architecture.md`
- ADR: `docs/adr/`
- 회고: `final-retrospective.md`